ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

PRINCIPLES OF COMPUTER PROGRAMMING

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 3 Principles of Computer Programming paper 1 · practice a.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code answers may be in any language or pseudocode. Mark the logic, not the syntax. The ticks show where marks are given; accept other correct solutions.

SECTION A

QUESTION 1

1.1 1.1.1 A ✓ 1.1.2 D ✓ 1.1.3 B ✓ 1.1.4 D ✓ 1.1.5 B ✓ 1.1.6 C ✓ 1.1.7 A ✓ 1.1.8 B ✓ 1.1.9 D ✓ 1.1.10 A ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 True ✓ 1.2.3 True ✓ 1.2.4 False ✓ 1.2.5 True ✓ 1.2.6 True ✓ 1.2.7 True ✓ 1.2.8 True ✓ 1.2.9 False ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 B ✓ 1.3.2 C ✓ 1.3.3 I ✓ 1.3.4 A ✓ 1.3.5 G ✓ 1.3.6 E ✓ 1.3.7 F ✓ 1.3.8 H ✓ 1.3.9 J ✓ 1.3.10 D ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 A compiler translates the whole program into machine code before it runs ✓; an interpreter translates and runs one line at a time ✓. Compiler advantage: the program runs faster / can be distributed without the source code ✓. Interpreter advantage: errors are found line by line, easier to test and debug ✓. (4)

2.2 (a) 2 ✓ (b) ababab ✓ (c) 3.7 ✓ (d) 7 ✓ (4)

2.3 (age >= 18 and age <= 35) ✓✓ and hasID == True ✓ (accept 18 <= age <= 35 and hasID) (3)

2.4 Any THREE: start with a letter, not a number ✓; no spaces (use camelCase or underscores) ✓; do not use reserved words (e.g. if, while) ✓; use meaningful names; names are case-sensitive (3)

2.5 Any TWO ✓✓: procedural; object-oriented; event-driven; functional. Description of ONE ✓, e.g. object-oriented – the program is built from objects that contain data (attributes) and methods; event-driven – code runs when events such as button clicks happen (3)

[17]

QUESTION 3

3.1 Example:

```
mark = int(input("Enter mark: "))       ✓
if mark < 0 or mark > 100:              ✓
    print("Invalid")                     ✓
elif mark >= 80:                         ✓
    print("Distinction")
elif mark >= 70:                         ✓
    print("Merit")
elif mark >= 50:                         ✓
    print("Pass")
else:                                    ✓
    print("Fail")                        ✓ correct outputs
```

(8)

3.2.1 Multiples of 3 from 1 to 10: 3 + 6 + 9 ✓✓ → output 18 ✓ (3)

3.2.2 n would never change ✓, so the condition stays True and the loop runs forever (infinite loop) ✓. (2)

3.2.3 Example:

```
total = 0
for n in range(1, 11):        ✓✓
    if n % 3 == 0:
        total = total + n
print(total)                  ✓
```

(3)

3.3 Example:

```
attempts = 0                                    ✓
pin = 0
while pin != 4321 and attempts < 3:             ✓✓
    pin = int(input("Enter PIN: "))             ✓
    attempts = attempts + 1                     ✓
if pin == 4321:
    print("Access granted")
else:
    print("Card blocked")                       ✓
```

(6)

[22]

QUESTION 4

4.1.1 36 ✓ (1)

4.1.2 Example:

```
highest = temps[0]                          ✓
total = 0                                   ✓
for t in temps:                             ✓
    if t > highest:                         ✓
        highest = t
    total = total + t                       ✓
average = total / len(temps)                ✓
print("Highest:", highest, "Average:", round(average, 1))   ✓
# Highest 36, average 221 / 7 = 31.6
```

(7)

4.1.3 count = 0; for t in temps: if t > 30: count = count + 1 ✓✓; print(count) → 4 ✓ (31, 34, 36, 33) (3)

4.2 A one-dimensional array stores values in a single row (one index) ✓; a two-dimensional array stores values in rows and columns (two indexes) ✓; example: marks of several students in several subjects / a seating plan / a timetable ✓. (3)

[14]

QUESTION 5

5.1 Example:

```
def calcVAT(price):              ✓ (name and parameter)
    return price * 1.15          ✓✓ (calculation and return)

result = calcVAT(200)            ✓ (call with argument)
print("Price incl. VAT: R", result)   ✓ (display)
# Output: 230.0                  ✓
```

(6)

5.2 Any THREE: code can be reused without rewriting it ✓; programs are easier to read and understand ✓; easier to test and debug each part separately ✓; teams can work on different parts; changes are made in one place (3)

5.3 Line 1: input is not converted to a number ✓ – mark1 = int(input("Mark 1: ")) ✓; Line 3: missing closing bracket ✓ – add) at the end ✓; Line 4: only mark3 is divided by 3 (order of operations) ✓ – average = (mark1 + mark2 + mark3) / 3 ✓. (Also accept line 5: cannot join a string and a number – use str(average) or a comma.) (6)

5.4 Line 3 – syntax error; line 4 – logic error; line 1 (or line 5) – runtime error (type error) ✓✓ (all three correct = 2; two correct = 1) (2)

[17]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE